

10.1 Debugging

- Debugging is the process of identifying and resolving errors (bugs) in your C code that prevent it from functioning as expected.

10.1.1 Common Debugging Practices

- **Code review** : Carefully examine your code line by line for syntax, typos and logical mistakes. Look for areas where you missed semicolons, used incorrect operators or made mistakes in variable usage. It can catch subtle errors that automated tools might miss; however, it is time-consuming.
- **Print statements (printf)** : Strategically placing printf statements throughout your code allows you to print the values of variables at different points in your program's execution. This can help you verify if variables are being assigned and modified as expected. It is simple and effective for small programs; however, it can be tedious and messy for large programs.
- **Debuggers** : Utilize debuggers like GDB (GNU Debugger) to step through your code line by line, inspect variable values and set breakpoints to pause execution at specific points. This allows you to examine the program's state in detail and identify issues.
- **Modular testing** : Break down your code into smaller, well-defined functions. Test these functions independently first to isolate potential problems before integrating them into the larger program.
- **Error handling** : Implement proper error handling mechanisms to catch issues like file access errors or division by zero. This can help prevent your program from crashing and provide more informative messages about what went wrong.
- **Rubber duck debugging** : Sometimes, explaining your code to an imaginary listener (like a rubber duck) can help you identify your own mistakes or areas needing improvement through the process of verbalization.

10.1.2 Debugging Tools

1. GDB (GNU Debugger)

- **Description** : GDB is a powerful command-line debugger for C (and C++) programs. It allows you to step through your program line by line, inspect variables and control the execution flow to pinpoint errors (bugs) in your code.
- **Key features of GDB** :
 - **Line-by-line execution** : You can use GDB to execute your program one line at a time. This allows you to examine the state of your program after each line, helping you identify where issues might be arising.

- **Breakpoints** : Set breakpoints at specific lines in your code where you want GDB to pause execution. This allows you to examine the values of variables and the program's state at those points.
- **Variable inspection** : GDB allows you to view the values of variables at any point during execution. This is crucial for verifying if variables are being assigned and modified as expected.
- **Stack trace** : When your program crashes, GDB can provide a stack trace that shows the sequence of function calls that led to the crash. This can help you identify the root cause of the problem.
- **Memory examination** : GDB allows you to inspect the contents of memory locations, which can be helpful for debugging memory-related errors like buffer overflows.
- **Command interface** : GDB offers a rich command interface that lets you perform various debugging operations like setting breakpoints, examining variables and continuing or stopping program execution.
- **Pros :**
 - Comprehensive debugging capabilities.
 - Widely used and well-documented.
- **Cons :**
 - Using the command-line interface can be scary for beginners.

Integrated Development Environments (IDEs)

- **Description** : Integrated Development Environments (IDEs) offer built-in debuggers that provide a more user-friendly interface compared to the command-line GDB for debugging C programs.
- **Examples** : Eclipse, Code::Blocks, CLion, Visual Studio Code (with extensions)
- **Features**
 - **Graphical interface** : IDE debuggers typically have a Graphical User Interface (GUI) that allows you to :
 - Set breakpoints by clicking on lines of code.
 - View variable values in a dedicated window.
 - Step through code line by line using buttons or keyboard shortcuts.
 - Explore the call stack visually.
 - **Integration with source code** : IDE debuggers are tightly integrated with your source code editor. This allows you to :
 - See the current line of execution highlighted in your code editor.
 - Easily navigate between source code and debugger windows.
 - Set breakpoints directly in the code editor.

- **Visualization tools** : Some IDEs offer additional visualization tools for debugging, such as :
 - Memory viewers to inspect memory allocations and potential leaks.
 - Thread debuggers for debugging multithreaded applications.
- **Pros :**
 - **Easier to learn** : The visual interface and integration with source code can make IDE debuggers easier to learn and use compared to the command-line GDB.
 - **Faster debugging workflow** : The graphical interface and features like breakpoint setting and variable inspection can streamline the debugging process.
 - **Suitable for beginners** : IDE debuggers are a good starting point for programmers new to debugging C programs.
- **Cons :**
 - **Less powerful** : While user-friendly, IDE debuggers might not offer the same level of control and advanced features as GDB.
 - **IDE-specific** : Debugger functionality can vary depending on the specific IDE you are using.
 - **Command-line knowledge still useful** : Understanding basic GDB commands can be beneficial even when using an IDE debugger.

3. Valgrind

- **Description** : Valgrind is a powerful memory debugging and profiling tool specifically designed to help you identify memory-related errors in C programs.
- **Features** : Valgrind excels at detecting various memory-related errors in C programs, including :
 - **Memory leaks** : When allocated memory is not freed when it's no longer needed, causing a gradual increase in memory usage.
 - **Use-after-free errors** : Attempting to access memory that has already been freed, potentially leading to crashes or unexpected behavior.
 - **Buffer overflows and underflows** : Accessing memory outside the allocated buffer size, which can overwrite other variables or crash the program.
 - **Invalid memory accesses** : Trying to access memory that hasn't been allocated or is not accessible to the program.
- **Pros :**
 - **Early detection** : Valgrind can help you catch memory errors early in the development process, preventing crashes and unexpected behavior in production environments.
 - **Detailed reports** : The reports generated by Valgrind provide valuable information to understand the nature of the memory error and its location in your code.

- **Complements other debuggers** : Valgrind works well alongside other debuggers like GDB. While GDB can help with general debugging, Valgrind specifically focuses on memory-related issues.
- **Cons** : Slows down program execution.

4. Static code analyzers

- **Description** : Tools that analyze your code without executing it to find potential errors and bugs. They scan for patterns, coding practices, and potential constructs that might lead to errors.
- **Examples** : Splint, cppcheck.
- **Pros** :
 - **Early detection** : SCAs can catch potential issues early in the development process, saving time and effort compared to debugging runtime errors later.
 - **Improved code quality** : By identifying and addressing potential problems, SCAs can help you write more robust and secure C programs.
- **Cons** : Might produce false positives.

5. Electric fence

- **Description** : eFence is a memory debugger that helps detect memory-related errors in C programs, such as :
 - Accessing memory outside the allocated bounds (buffer overflows, underflows).
 - Using memory after it has been freed (dangling pointers).
- **Pros** : Simple to use and effective for catching memory errors.
- **Cons** : Adds significant overhead and can slow down the program.

6. SANitizers

- **Description** : Sanitizers, specifically AddressSanitizer (ASan), are powerful debugging tools for C programming that help detect memory-related errors.
- **Function** : ASan is an instrumentation tool that integrates with your compiler to detect memory access errors in your C program. It works by :
 - Inserting additional code into your program during compilation.
 - Tracking memory allocations and deallocations.
 - Monitoring all memory accesses to ensure they stay within valid bounds.
- **Error Detection** : ASan can identify various memory-related errors, including :
 - **Buffer overflows and underflows** : Accessing memory beyond the allocated buffer size.
 - **Use-after-free** : Attempting to use memory that has already been freed.
 - **Stack overflows and underflows** : Accessing memory outside the valid stack boundaries.

- **Pros** : ASan helps prevent crashes and unexpected program behavior caused by memory errors. It can be a valuable tool in the development process to ensure code stability.
- **Cons** : Performance overhead.

7. Perf

- **Description** : Perf is a powerful tool for performance analysis in C programming, not strictly a debugger for identifying logical errors. However, it can be incredibly helpful in debugging performance bottlenecks in your C programs.
- **Function** : Perf focuses on analyzing the performance of your program. It allows you to profile :
 - **CPU usage** : Identify which parts of your code consume the most CPU cycles.
 - **Function calls** : Track how often functions are called and how much time they take to execute.
 - **Memory usage** : Analyze memory allocation patterns and potential memory bottlenecks.
 - **Cache misses** : Understand how efficiently your program utilizes the CPU cache.
- **Pros** : Helps optimize performance
- **Cons** :
 - Requires understanding of performance metrics
 - Can introduce slight overhead to your program's execution.
 - By leveraging these tools, you can effectively debug and improve your C programs. Each tool has its strengths and is suited for different types of debugging tasks, from memory issues and performance bottlenecks to logical errors and code analysis. Combining multiple tools and techniques often yields the best results in maintaining robust and error-free C code.

Review Questions

1. What is debugging ?
2. Explain commonly used debugging practices.
3. Explain any two debugging tools.

10.2 Testing

- Software testing is a process used to ensure that a software application works correctly and meets the user's expectations. It involves checking the software for errors, bugs and other issues to ensure it performs as expected.
- Besides finding faults, software testing also helps to improve the software's efficiency, usability and accuracy. To test software, a specific format called a **test case** is used.

10.2.1 Writing Test Cases

A test case is a specific format used in software testing to see if an application or software is working correctly. It includes a set of conditions to be checked to determine if the software produces the expected results. In simpler terms, it checks if the actual output matches the expected output. A test case typically includes various details such as an ID, the condition to be tested, steps to follow, input data, expected result, actual result, status (pass or fail) and any remarks.

10.2.1.1 Parameters of a Test Case

- A test case typically includes the following parameters :
 1. **Test case ID** : A unique identifier for the test case to differentiate it from others.
 2. **Description** : A brief explanation of what the test case is intended to verify.
 3. **Preconditions** : Any setup or initial conditions must be met before the test is executed.
 4. **Test steps** : A detailed, step-by-step list of actions to perform the test.
 5. **Input data** : Specific data to be used during the test.
 6. **Expected result** : The outcome should occur if the software functions correctly.
 7. **Actual result** : The actual outcome observed when the test is executed. This is filled in, after running the test.
 8. **Status** : Indicates whether the test passed or failed comparing the expected and actual results.
 9. **Remarks / Comments** : Any additional information, observations or notes about the test.

10.2.1.2 Example of a Test Case

- Let us see an example to illustrate these parameters :
 1. **Test case ID** : TC001
 2. **Description** : Verify that a user can log in with valid credentials.
 3. **Preconditions** : The user is registered and has a valid username and password.
 4. **Test steps** :
 - a. Open the login page.
 - b. Enter the valid username and password.
 - c. Click the login button.
 5. **Input data** : Username : **testuser**, Password : **password123**.
 6. **Expected result** : The user is successfully logged in and redirected to the dashboard.

7. **Actual result** : (Filled in after the test is executed.)
8. **Status** : (Marked as pass or fail after the test is executed.)
9. **Remarks / Comments** : (Any additional notes or observations.)

10.2.1.3 When Do We Write Test Cases ?

- Test cases are written in different situations to ensure the software meets all requirements and works correctly. Here are the key times when test cases are created :

1. Before development :

- **Purpose** : Identify the requirements of the product/software.
- **Benefit** : Helps clarify what needs to be developed and tested later.
- **Example** : Writing test cases for a login feature before the coding starts.

2. During development :

- **Purpose** : Test parts of the software as they are being developed.
- **Benefit** : Allows immediate testing of each component as it is created.
- **Example** : Writing test cases for a new module while the module is being coded.

3. After development :

- **Purpose** : Validate the functionality of the completed product/software or a specific feature.
- **Benefit** : Ensures the entire product or new feature works as intended before launch.
- **Example** : Writing test cases for a new feature after it has been implemented but before it is released.

10.2.1.4 Importance and Advantages of Writing Test Cases

- Test cases are a crucial part of software engineering, defining how testing is carried out to ensure software functionality. Here are some of the main reasons and advantages for writing test cases :
1. **Makes sure your program meets customer expectations** : Test cases are like a checklist to see if your program does what it's supposed to do, according to what users need.
 2. **Catches errors early** : By testing different situations, you can find problems before they become a bigger issue later.
 3. **Saves time in the long run** : Test cases help you avoid making the same bug fixes over and over again.

4. **Makes testing easier** : Clear test cases act like instructions for anyone who needs to test your program, ensuring consistency in how it's tested.
5. **Helps improve your program** : Thinking about different scenarios while writing test cases can inspire you to improve your program's features.
6. **Useful for updates** : When you make changes to your program, test cases help you check if everything still works as expected.
7. **Narrow down software updates** : Test cases help to narrow down the software needs and required updates.

10.2.1.5 Best Practices for Writing Test Cases

1. **Simple and clear** : Test cases should be concise, clear and easy to understand for everyone.
2. **Unique tests** : Each test case should be unique and focus on a different scenario. Avoid redundancy.
3. **No assumptions** : Don't make assumptions about data or features that don't exist in your program.
4. **Traceable** : Test cases should be traceable for future reference, making them easy to follow up on.
5. **Different input data** : Consider all sorts of data inputs, both valid and invalid, to test your program thoroughly.
6. **Clear module names** : Use self-explanatory names for modules in test cases.
7. **Concise descriptions** : Keep test case descriptions short and to the point, but informative enough to give a basic overview.
8. **Think of all conditions** : Try to cover as many conditions (different program states) as possible in your tests.
9. **Meeting needs** : Make sure your test cases align with the requirements set by users or clients.
10. **Repetitive results** : Write test cases so they produce the same result each time.
11. **Alternative testing methods** : If testing all conditions is difficult, use different testing techniques to cover various aspects of the program.
12. **User perspective** : Create test cases from the end user's point of view, ensuring they meet user needs.
13. **Unique IDs** : Use unique IDs for test cases, following a naming convention for clarity.
14. **Proper preconditions and postconditions** : Clearly mention any preconditions and postconditions.

15. **Reusable test cases** : Ensure test cases can be updated and reused when the code changes.
 16. **Exact expected outcome** : Specify the exact expected result for each test step.
- By following these practices, you can write effective test cases that ensure your program functions as expected and meets user requirements.

10.2.1.6 Test Case Management Tools

- Test management tools help manage test cases more efficiently than traditional methods.
- These automated tools save time and effort for testers.
- They often include features like advanced dashboards, easier bug and progress tracking, custom test case templates, test case integration, inviting guest testers, managing team requirements and plans and more.
- Some popular test case management tools are :
 1. **Testpad** : Testpad is a simple tool designed to make test case management easier. Its motto is to find bugs that matter. Key features include :
 - Manual testing
 - Test case and software reports
 - Drag-and-drop functionality
 - Inviting guest testers via email
 - Building custom templates.
 2. **TestCaseLab** : TestCaseLab is user-friendly and integrates easily with bug trackers. Key features include :
 - Custom test cases
 - Test runs
 - Test case integrations
 - Test plans
 - Tags and priority settings for test cases
 - Search by name, description, tags, etc.
 3. **TestRail** : TestRail aims to simplify test case management and streamline software testing processes, providing greater visibility into QA. Key features include :
 - Management of test cases, plans and runs
 - Increased test coverage
 - Real-time insights into QA progress.

4. **TestLodge** : TestLodge helps teams manage their requirements, test plans, test cases and test runs all in one place with no user limit. Key features include :
- Test plans
 - Test runs
 - A dashboard
 - A test suite.
- These tools enhance efficiency and organization in test case management, making the testing process smoother and more effective.

10.2.1.7 Types of Test Cases

1. Functionality test case :

- Determines if the software's interface works smoothly with the rest of the system and its users.
- Uses black box testing, checking everything externally rather than internally.

2. Unit test case :

- Tests individual parts or units of the software.
- Each unit is tested separately with a different test case for each one.

3. User Interface (UI) test case :

- Tests every component of the user interface that the user interacts with.
- Ensures that the UI components meet user requirements.

4. Integration test case :

- Tests the combination of all software units.
- Ensures that each component and unit works together without issues.

5. Performance test case :

- Determines the response time and overall effectiveness of the software.
- Ensures the application can handle real-world expectations.

6. Database test case :

- Also known as back-end testing, checks everything related to the database.
- Includes testing tables, schema, triggers, etc.

7. Security test case :

- Ensures the application restricts actions and permissions as necessary.
- Focuses on encryption and authentication to protect and safeguard data.

8. Usability test case :

- Also known as a user experience test case, checks how user-friendly the software is.
- Designed by the user experience team and executed by the testing team.

9. User acceptance test case :

- Prepared by the testing team but executed by the user/client.
- Ensures the software works in a real-world environment.

Review Questions

1. What is software testing ?
2. What is test case ?
3. What are the parameters of test cases ?
4. Give one example of test case.
5. When do we write test cases ?
6. State the importance and advantages of test cases.
7. Discuss best practices for writing test cases.
8. Write a note on test case management tools.
9. Discuss various types of test cases.

10.3 Unit Testing

- **Definition :** Unit testing is a software testing method where individual components or units of the software are tested in isolation from the rest of the system. Each unit, typically the smallest testable part of an application like functions, methods or classes is tested independently to ensure it performs as expected.
- **Purpose :** The main goal of unit testing is to validate that each unit of the software performs correctly. It helps identify bugs and issues at an early stage in the development process, making it easier to fix them before they impact other parts of the system.

Key features of unit testing

1. **Isolation :** Units are tested in isolation from the rest of the application to ensure that they work independently.
2. **Early detection of errors :** Helps detect errors and bugs early in the development cycle, reducing the cost and effort of fixing issues later.
3. **Documentation :** Unit tests can serve as documentation for the code, providing examples of how units are expected to behave.

4. **Refactoring support** : With unit tests in place, developers can confidently refactor code, knowing that the tests will catch any unintended changes.

Steps in unit testing

1. **Writing test cases** : This involves creating instructions that specify what input to give a function or code unit, what actions it should take and what output it should produce.
2. **Using a testing framework** : Most programming languages have testing frameworks that provide tools and structures to write and run unit tests efficiently.
3. **Running tests** : The testing framework executes the test cases and compares the actual output of your code with the expected output you defined in the test cases.
4. **Fixing bugs** : If a test fails, it indicates an error in your code. You will need to debug and fix the code to make the test pass.

Benefits of unit testing

1. **Improves code quality** : By testing individual units, developers can ensure each part of the code is working correctly, leading to higher overall code quality.
2. **Reduces bugs in later stages** : Catching and fixing bugs early means fewer bugs are carried forward to later stages of development, reducing the complexity and cost of fixing them.
3. **Simplifies integration** : With units tested independently, integration of components is smoother since each part is already verified to work correctly.
4. **Improves code maintainability** : Well-tested code is easier to understand and modify in the future.

Review Questions

1. What is unit testing ? State its purpose.
2. List the key features of unit testing.
3. What are the steps carried out in unit testing ?
4. What are the benefits of unit testing ?